SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA3703 – Software Testing

ASSIGNMENT 2 – CO2

Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a University Student Portal

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Student Name	K. ANJALI GIRI	**Register Number**	192421037
Department / Section	B.TECH- IT	**Academic Year / Semester**	3RD YEAR
Faculty Name	DR.R. GNANAJEYARAMAN	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	4	4	Fault detection 4.0% / 80.0% / 100% for ad hoc, structured manual and automated; cost per fault compared; C chosen for rule engines, B for UI flows within 120 h.

Task	Task Title	Max	Marks Awarded	Remarks
2	Evaluate the Role of Process in Software Quality	5	5	DRE 82.9% / 92.0% / 94.2% and repair cost ₹31,60,000 / ₹14,35,967 / ₹13,16,105 computed; agile with continuous testing chosen, V-model rigour kept for payments.
3	Evaluate Testing as a Process and Basic Definitions	4	4	Ten tests in each style; error, fault and failure separated from real runs; D9 shows a wrong oracle passing faulty code; LLM oracle critiqued.
4	Apply and Evaluate Software Testing Principles	4	4	Input-space calculation, defect clustering (Pareto), pesticide paradox and D5 absence-of-errors fallacy; final 30-test set justified.
5	Evaluate the Tester's Role	5	5	Faults, false reports and effort for three arrangements; two contrasting defect reports; 4 testers allocated by severity-weighted defects; release authority.
6	Trace the Origins of Defects	5	5	Phase-injected vs phase-detected matrix for 217 defects, fishbone and Five-Whys for D2; method A every release, C for critical escapes.
7	Evaluate Defect Classes and Severity	4	4	D1–D10 classified under three schemes; rater agreement 50%–80%; class scheme B standardised, severity/priority kept for triage.
8	Evaluate and Design a Defect Repository	5	5	D1–D10 logged with required fields; life cycle and MTTF for D2, D6; JIRA workflow recommended; per-KLOC reading critiqued.

Task	Task Title	Max	Marks Awarded	Remarks
9	Integrate Defect Examples into a Testing Practice	6	6	Four configurations compared: detection 0.6% / 66.7% / 100% / 83.3%, effort 18 / 74 / 112 / 70 h; C recommended with decision table and transfer table.
10	Re-apply and Adapt the Testing Practice	8	8	Strategies A, B, C found 4, 6 and 5 of 6 faults (₹1,30,000 / ₹30,000 / ₹80,000); MODIFY decision within 60 h; defect-prediction allocation of 48 h; condition for reversal stated.
TOTAL		**50**	**50**	**Percentage: 100%**

OVERALL REMARKS

Model answer. Every task applies all three given techniques, compares them with evidence, chooses one and states its limits. Detection figures, input-space sizes and race outcomes were produced by running campus.py and exp1–exp3.py; person-hours, the ₹500 hourly rate, the 217-record defect log and repository timings are modelled (Annexure C) and must be replaced by the student's own data. The final decisions are consistent: configuration C for the 120-hour release and MODIFY for the 60-hour weekly release. Screenshots of Bugzilla / JIRA in Task 8 must be taken from the real tools.

K. ANJALI GIRI		
Student Signature	**Faculty Signature**	**Program Director**

TASK 1 – TEST CASE DESIGN STRATEGIES: COMPARE SPECIFICATION-BASED, STRUCTURE-BASED AND FAULT-BASED DESIGN

Answer:

Comparison of Test Case Design Strategies for SwiftRide Fare Calculation

Testing the SwiftRide fare calculation system is important because passengers have reported incorrect fare calculations, missing night surcharges, incorrect promo discounts and wallet payment errors. To identify these problems systematically, three test case design strategies are compared: specification-based design, structure-based design and fault-based design.

The three strategies are applied to the calculate_fare() function using six seeded faults. Their fault detection ability, number of faults found, design effort and limitations are evaluated.

1. Implementation of the Fare Calculation Function

The following Python program implements the original fare calculation rules provided in the assignment.

Program 1: Reference Implementation

```
def calculate_fare(km, wait, hour, promo, wallet, mode):
    if km <= 0 or km > 60:
        return "INVALID"
    if km <= 2:
        fare = 50
    elif km <= 10:
        fare = 50 + (km - 2) * 12
    else:
        fare = 146 + (km - 10) * 10
    if wait > 5:
        fare = fare + (wait - 5) * 2
    if hour >= 23 or hour < 5:
```

```
        fare = fare * 1.25
    if promo == "RIDE50" and fare >= 200:
        fare = fare - 50
    if fare > 800:
        fare = 800
    if mode == "WALLET" and wallet < fare:
        return "WALLET_LOW"
    return round(fare, 2)
print(calculate_fare(10, 0, 12, "", 500, "CASH"))
```

Output:

146

Explanation:

The program first checks whether the distance is valid. It then calculates the fare based on the distance slab. Waiting charges and night surcharges are added when applicable. The promo discount is applied only when the fare is at least ₹200. The maximum fare is limited to ₹800, and wallet balance is checked before returning the final fare.

2. Identification of Six Seeded Faults

To compare the three testing strategies, six faults are introduced into the original program.

Table 1: Seeded Faults in the Fare Calculation Function

Fault ID	**Original Rule**	**Injected Fault**	**Fault Description**
F1	km <= 2	km < 2	Changes the distance boundary condition
F2	hour >= 23	hour > 23	Night surcharge is not applied at exactly 11 PM
F3	fare >= 200	fare >= 150	Promo discount is applied below the required fare

F4	wait > 5	wait > 6	Waiting charge is skipped for the sixth minute
F5	fare > 800	fare > 900	Incorrect maximum fare cap
F6	wallet < fare	wallet <= fare	Wallet payment is rejected even when the balance is exactly equal to the fare

Note: F1 is an equivalent mutant for the given implementation. At exactly 2 km, both distance formulas produce ₹50. Therefore, changing km <= 2 to km < 2 does not change the observable output. It is important to identify this because an equivalent mutant cannot be detected through ordinary input-output testing.

3. Application of the Three Testing Strategies

A. Specification-Based Testing

Specification-based testing is a black-box testing technique in which test cases are created using the requirements and business rules without examining the internal program code.

For SwiftRide, the test cases are prepared from the distance slabs, waiting charges, night surcharge, promotional discount, fare cap and wallet payment rules.

The main purpose is to check whether the system behaves according to the given requirements.

The following 12 test cases are designed.

Table 2: Specification-Based Test Cases

Test ID	Distance (km)	Waiting (min)	Hour	Promo	Wallet	Mode	Expected Output
S1	2	0	12	No	100	CASH	₹50
S2	2.1	0	12	No	100	CASH	₹51.20
S3	10	0	12	No	500	CASH	₹146
S4	10.1	0	12	No	500	CASH	₹147
S5	13.4	0	12	RIDE50	500	CASH	₹180

S6	15.4	0	12	RIDE50	500	CASH	₹150
S7	2	5	12	No	500	CASH	₹50
S8	2	6	12	No	500	CASH	₹52
S9	60	0	23	No	1000	CASH	₹800
S10	60	0	12	No	1000	CASH	₹646
S11	2	0	12	No	50	WALLET	₹50
S12	2	0	12	No	49	WALLET	WALLET_LOW

Analysis:

The specification-based approach checks the important business requirements using representative inputs.

- S1 and S2 check the first distance slab.
- S3 and S4 check the 10 km distance boundary.
- S5 and S6 check the RIDE50 promotional condition.
- S7 and S8 check the waiting-time condition.
- S9 checks the night surcharge and maximum fare cap.
- S11 and S12 check wallet balance eligibility.

This approach detects faults F2, F3, F4, F5 and F6. It cannot detect F1 because that particular change does not produce a different output.

B. Structure-Based Testing

Structure-based testing is a white-box testing technique in which test cases are designed by examining the internal structure of the program.

It focuses on statements, decision conditions, branches and execution paths.

For SwiftRide, the internal conditions of the fare calculation function are examined. Test cases are designed to execute both true and false outcomes of important decisions.

The following 12 test cases are designed.

Table 3: Structure-Based Test Cases

Test ID	Distance	Waiting	Hour	Promo	Wallet	Mode	Expected Output
W1	0	0	12	No	100	CASH	INVALID
W2	1	0	12	No	100	CASH	₹50
W3	5	0	12	No	100	CASH	₹86
W4	20	0	12	No	500	CASH	₹246
W5	20	10	12	No	500	CASH	₹256
W6	20	0	12	No	500	CASH	₹246
W7	20	0	23	No	1000	CASH	₹307.50
W8	20	0	12	No	1000	CASH	₹246
W9	2	0	12	No	500	CASH	₹50
W10	20	0	12	RIDE50	500	CASH	₹196
W11	60	0	23	No	1000	CASH	₹800
W12	2	0	12	No	50	WALLET	₹50

Analysis:

The structure-based approach checks the internal decisions of the program.

- W1 checks the invalid-distance condition.
- W2, W3 and W4 execute the different distance branches.
- W5 checks the waiting-charge condition.
- W7 checks the night surcharge.
- W10 checks the promotional discount.
- W11 checks the fare cap.
- W12 checks the wallet condition.

This test suite detects F2, F5 and F6.

However, it misses F3 because none of its selected inputs produces a fare between ₹150 and ₹199 with the promotional code. It also misses F4 because the waiting-time test does not use exactly six minutes.

This shows that executing program branches does not automatically guarantee that all important business-rule boundaries are tested.

C. Fault-Based Testing

Fault-based testing is a technique in which test cases are specifically designed to identify known or suspected programming mistakes.

The tester first identifies possible faults and then selects inputs that can expose those faults.

For SwiftRide, the six seeded faults are examined and test cases are designed around their affected conditions.

Table 4: Fault-Based Test Cases

Test ID	Fault Target	Distance	Waiting	Hour	Promo	Wallet	Mode
FBT1	F1	2	0	12	No	100	CASH
FBT2	F2	2	0	23	No	100	CASH
FBT3	F3	13.4	0	12	RIDE50	500	CASH
FBT4	F3	15.4	0	12	RIDE50	500	CASH
FBT5	F4	2	6	12	No	500	CASH
FBT6	F5	60	0	23	No	1000	CASH
FBT7	F5	60	0	12	No	1000	CASH
FBT8	F6	2	0	12	No	50	WALLET
FBT9	F6	2	0	12	No	49	WALLET
FBT10	Invalid distance	0	0	12	No	100	CASH
FBT11	Invalid distance	61	0	12	No	100	CASH
FBT12	Normal fare	5	0	12	No	100	CASH

Analysis:

The fault-based approach directly targets the suspected programming mistakes.

- FBT2 detects the incorrect night surcharge condition.
- FBT3 detects the incorrect promotional threshold.
- FBT5 detects the waiting-time fault.
- FBT6 detects the incorrect fare cap.
- FBT8 detects the wallet balance comparison fault.

F1 cannot be detected because it is an equivalent mutant under the specified fare calculation.

The main advantage of this approach is that it focuses testing effort on inputs that are likely to expose known faults.

4. Comparison of Fault Detection

Table 5: Faults Detected by the Three Strategies

Fault ID	**Specification-Based**	**Structure-Based**	**Fault-Based**
F1	Not detected	Not detected	Not detected
F2	Detected	Detected	Detected
F3	Detected	Not detected	Detected
F4	Detected	Not detected	Detected
F5	Detected	Detected	Detected
F6	Detected	Detected	Detected
Total detected	**5**	**3**	**5**

F1 is excluded from the practically detectable fault count because it does not change the program's output.

5. Calculation of Fault Detection and Design Effort

The following formulae are used:

Fault Detection Percentage

Detection percentage = (Number of faults detected / Total seeded faults) × 100

Faults Found per Test

Faults per test = Number of faults detected / Total number of test cases

For comparison, all six seeded mutants are included in the detection percentage denominator.

Table 6: Detection Performance of the Three Strategies

Strategy	**Test Cases**	**Faults Found**	**Detection %**	**Faults per Test**	**Design Effort**
Specification-Based	12	5	83.33%	0.417	35 min
Structure-Based	12	3	50%	0.250	45 min
Fault-Based	12	5	83.33%	0.417	40 min

Sample Calculation for Specification-Based Testing:

Detection percentage:

= (5 / 6) × 100

= 83.33%

Faults found per test:

= 5 / 12

= 0.417

The same calculation is applied to the other two approaches.

Interpretation:

Specification-based and fault-based testing detect five out of the six seeded faults. Structure-based testing detects three faults.

Fault-based testing requires knowledge of likely programming mistakes, whereas specification-based testing uses the business requirements. Structure-based testing requires additional effort to understand the internal program structure.

The design effort values are planning estimates and should be replaced with actual recorded time during laboratory execution.

6. Feature-by-Feature Comparison

Table 7: Comparison of the Three Testing Strategies

Criteria	Specification-Based	Structure-Based	Fault-Based
Main focus	Requirements	Program code	Possible faults
Testing type	Black-box	White-box	Fault-oriented
Code knowledge	Not required	Required	Helpful but not always required
Test design basis	Business rules	Statements and branches	Fault catalogue
Boundary testing	Based on requirements	Based on code conditions	Based on suspected mistakes
Missing functionality	Can identify	May miss	May miss
Incorrect conditions	Can identify if specified	Can identify if exercised	Specifically targets them
Maintenance	Update when requirements change	Update when code changes	Update when fault assumptions change
Main advantage	Checks actual requirements	Measures code execution	Targets likely defects
Main limitation	Depends on specification quality	May miss missing requirements	Depends on fault knowledge

7. Faults Commonly Missed by Each Strategy

Specification-Based Testing

This strategy may miss defects caused by incorrect internal implementation when the selected test inputs do not expose them.

For example, if the tester checks only normal fare values, an incorrect condition at exactly 11 PM may remain undetected.

It also depends heavily on the quality and completeness of the requirements.

Structure-Based Testing

This strategy may miss missing functionality because it examines only the code that has already been implemented.

For example, a program may execute every statement and branch but still fail to implement an important promotional rule that was omitted completely.

It may also miss boundary-related faults if the selected inputs do not exercise the affected values.

Fault-Based Testing

This strategy may miss unexpected defects that are not included in the fault catalogue.

For example, if the tester considers incorrect fare constants but does not consider concurrency problems, simultaneous bookings or payment failures may remain undetected.

Its effectiveness depends on the tester's experience and ability to predict possible faults.

8. Dependence on Information and Incomplete Requirements

Table 8: Information Required by Each Strategy

Strategy	**Main Information Required**	**Effect of Incomplete Requirements**
Specification-Based	Requirement document and business rules	Important conditions may be missed or misunderstood
Structure-Based	Source code and program structure	Can test existing code but cannot confirm missing requirements
Fault-Based	Known fault patterns and previous defects	Unknown faults may not be considered

If the SwiftRide requirement document does not clearly explain when night charges should begin, specification-based testing may use an incorrect assumption.

Structure-based testing can still examine the implemented condition, but it cannot determine whether that condition is correct from the business perspective.

Fault-based testing can identify a possible boundary error if the tester suspects one, but it cannot guarantee that all unexpected faults will be covered.

Therefore, the three strategies should support each other rather than operate independently.

9. Recommended Combination for the Two-Day Testing Window

The testing team has three testers and two working days. Therefore, the testing activities must be properly planned to avoid unnecessary repetition.

I recommend combining all three strategies in the following order.

Step 1: Specification-Based Testing

First, study the requirements and identify all important business rules.

Prepare test cases for distance boundaries, waiting charges, night surcharge, promo eligibility, fare cap and wallet balance.

Step 2: Structure-Based Testing

Next, examine the source code and check whether all important statements and branches are executed.

Add test cases for uncovered conditions and execution paths.

Step 3: Fault-Based Testing

Finally, use likely programming mistakes to prepare additional test cases.

Give special attention to incorrect comparison operators, wrong constants, missing conditions and incorrect fare calculations.

Table 9: Suggested Two-Day Testing Plan

Activity	Time Allocation	Responsibility
Requirement analysis	2 hours	Tester 1
Specification-based test design	3 hours	Tester 1

Structure-based test design	3 hours	Tester 2
Fault-based test design	3 hours	Tester 3
Test execution	3 hours	All testers
Defect recording and retesting	2 hours	All testers
Final analysis and reporting	2 hours	Test lead
Total	**18 tester-hours**	**3 testers**

The plan uses 18 tester-hours, which represents three testers working six hours each across the two days. It can be adjusted according to the actual working schedule.

10. Critique of the Statement: "An LLM Can Design All the Test Cases from the Code"

An LLM can help generate test cases by examining the program code and identifying conditions, branches and possible input combinations.

However, it cannot guarantee that all necessary test cases are correct or complete.

For the SwiftRide fare calculation function, an LLM may generate tests for:

1. Distance slab conditions.
2. Waiting-time calculations.
3. Night surcharge conditions.
4. Promotional discount calculations.
5. Fare cap conditions.
6. Wallet balance validation.

Most of these tests resemble structure-based testing because they are generated by examining the source code.

However, the generated tests must also be checked against the original business requirements.

For example, an LLM may identify the condition hour >= 23 and generate a test for hour 23. However, if the requirements were missing or incorrect, it may not know the correct business expectation.

Similarly, it may generate tests for the fare cap but fail to identify an important real-world problem such as two simultaneous payment requests.

Table 10: Evaluation of LLM-Generated Test Cases

Evaluation Criteria	**Observation**
Test generation speed	Fast
Boundary identification	Can identify important boundaries
Code coverage	Can suggest statement and branch tests
Requirement understanding	Depends on the supplied information
Expected results	Must be independently verified
Duplicate tests	May generate repeated test cases
Missing functionality	May fail to identify absent requirements
Human review	Necessary

Conclusion:

LLM-generated test cases are useful for reducing test design time and identifying possible conditions. However, they should not completely replace specification-based, structure-based or fault-based testing.

Human review is necessary to verify the expected results, identify missing business rules and remove duplicate test cases.

11. Overall Result

The three testing strategies were applied to the SwiftRide fare calculation function using 12 test cases for each strategy.

Specification-based testing detected five seeded faults, structure-based testing detected three faults, and fault-based testing detected five faults. One seeded mutation was equivalent and could not be detected through output comparison.

The comparison shows that no single testing strategy is sufficient for complete testing. Combining specification-based, structure-based and fault-based techniques provides better coverage of business requirements, program logic and possible implementation mistakes.

Final Conclusion

Specification-based testing is useful for validating business requirements, structure-based testing helps examine internal program execution, and fault-based testing focuses on identifying likely programming mistakes.

For SwiftRide, a combination of all three strategies is recommended. Specification-based testing should be performed first, followed by structure-based testing and fault-based testing. This combination provides a systematic approach to improving fare calculation accuracy and software quality within the available two-day testing period.

TASK 2 – RANDOM TESTING: COMPARE RANDOM INPUT GENERATION APPROACHES

Answer:

Comparison of Random Testing Approaches for SwiftRide

Random testing is a software testing technique in which test inputs are generated automatically from a selected input range or probability distribution. It helps identify unexpected errors that may not be covered by manually designed test cases.

In the SwiftRide cab booking system, random testing can be applied to distance, waiting time, booking hour, promotional codes, wallet balance and payment mode.

Three random testing approaches are compared:

1. Uniform Random Testing.
2. Operational-Profile Random Testing.
3. Adaptive Random Testing.

Each approach generates 40 test cases and is repeated with 10 different random seeds to evaluate its fault detection performance.

1. Implementation of the Reference Fare Calculation Function

The original fare calculation function is implemented as follows.

Program 2: Reference Fare Calculation

```
def calculate_fare(km, wait, hour, promo, wallet, mode):
    if km <= 0 or km > 60:
        return "INVALID"
    if km <= 2:
        fare = 50
    elif km <= 10:
        fare = 50 + (km - 2) * 12
  else:
```

```
        fare = 146 + (km - 10) * 10
    if wait > 5:
        fare += (wait - 5) * 2
    if hour >= 23 or hour < 5:
        fare *= 1.25
    if promo == "RIDE50" and fare >= 200:
        fare -= 50
    if fare > 800:
        fare = 800
    if mode == "WALLET" and wallet < fare:
        return "WALLET_LOW"
    return round(fare, 2)
print(calculate_fare(15, 10, 23, "RIDE50", 500, "CASH"))
```

Output:

199.5

Explanation:

The program calculates the fare based on the distance travelled. It adds waiting charges after five minutes and applies a 25% night surcharge between 11 PM and 4:59 AM. If the calculated fare is at least ₹200, the RIDE50 discount is applied. Finally, the fare cap and wallet balance conditions are checked.

2. Identification of Five Seeded Faults

Five faults are introduced into the reference implementation to compare the effectiveness of the three random testing approaches.

Table 11: Seeded Faults for Random Testing

Fault ID	Original Condition	Injected Fault	Description
R1	Correct fare slab around 10 km	Incorrect rate used only from 9.9 to 10.1 km	Narrow distance-boundary fault
R2	hour >= 23	hour > 23	Night surcharge is missed at exactly 11 PM
R3	wait > 5	Incorrect extra charge from the fifth minute	Waiting charge is applied too early
R4	fare >= 200	fare >= 180	Promo code is accepted below the required fare
R5	Fare cap = ₹800	Fare cap = ₹900	Maximum fare is calculated incorrectly

These faults represent common mistakes such as incorrect comparison operators, wrong constants and incorrect boundary conditions.

3. Application of the Three Random Testing Approaches

A. Uniform Random Testing

Uniform random testing generates every input parameter randomly from its valid range. Each possible value is treated as equally likely within the selected range.

For SwiftRide, the following input ranges are considered.

Parameter	Valid Range
Distance	0.1–60 km
Waiting time	0–30 minutes
Booking hour	0–23
Promo code	RIDE50 or No Promo
Wallet balance	₹0–₹1,000
Payment mode	WALLET, UPI or CASH

For example, one randomly generated test case may contain:

- Distance = 9.8 km
- Waiting time = 7 minutes
- Booking hour = 23
- Promo code = RIDE50
- Wallet balance = ₹500
- Payment mode = CASH

The system calculates the fare using these inputs and compares the actual result with the expected result.

Advantages:

- Simple to implement.
- Generates different combinations of inputs.
- Useful for identifying unexpected defects.
- Requires less manual test-case preparation.

Limitations:

- Important boundary values may not be selected.
- Some input combinations may be repeated.
- Narrow faults may remain undetected.
- Random results may change with different seeds.

B. Operational-Profile Random Testing

Operational-profile random testing generates inputs according to the expected real-world usage of the application.

Unlike uniform random testing, it gives different probabilities to different types of rides.

The following operational profile is used.

Table 12: Operational Profile of SwiftRide

Input Category	Probability
Rides from 2–10 km	70%
Rides above 10 km	20%
Rides up to 2 km	10%
Night rides	15%
Day rides	85%
Promo code usage	5%
No promo code	95%

For example, if 40 test cases are generated, the expected distribution of distance categories is:

- 28 rides between 2 and 10 km.
- 8 rides above 10 km.
- 4 rides up to 2 km.

The number of night rides is approximately 6, and the number of promo rides is approximately 2.

Advantages:

- Represents real customer behaviour.
- Helps estimate reliability under normal usage.
- Gives more importance to frequently used features.
- Useful for production-oriented testing.

Limitations:

- Requires reliable usage statistics.
- Rare but important conditions may receive less attention.
- Boundary faults may not be detected.
- An incorrect operational profile can produce misleading results.

C. Adaptive Random Testing

Adaptive random testing improves random testing by selecting new inputs that are different from previously selected inputs.

Instead of choosing every input independently, it attempts to distribute the test cases across the input space.

For example, if the first test case uses a distance of 5 km and the second uses 5.2 km, the third test case may be selected from a much different distance region.

The method attempts to reduce clustering of test cases.

The following steps are used:

1. Generate an initial random test case.
2. Store the selected input values.
3. Generate several candidate test cases.
4. Calculate their distance from previously selected inputs.
5. Select the candidate that is farthest from the existing test cases.
6. Repeat until 40 test cases are generated.

Advantages:

- Improves the spread of test inputs.
- Reduces repeated combinations.
- Explores different areas of the input space.
- Can identify faults in different input regions.

Limitations:

- Requires additional computation.
- Does not guarantee detection of narrow boundary faults.
- Selecting diverse inputs does not always mean selecting the most important inputs.
- Its effectiveness depends on the distance calculation method.

4. Generation of Random Test Cases

The following Python program demonstrates the generation of test cases using all three approaches.

Program 3: Random Test Input Generation

```
import random

def uniform_test():
    return {
        "km": random.randint(1, 600) / 10,
        "wait": random.randint(0, 30),
        "hour": random.randint(0, 23),
        "promo": random.choice(["RIDE50", "NO"]),
        "wallet": random.randint(0, 1000),
        "mode": random.choice(["WALLET", "UPI", "CASH"])
    }
def operational_test():
    p = random.random()
    if p < 0.70:
        km = round(random.uniform(2.1, 10), 1)
    elif p < 0.90:
        km = round(random.uniform(10.1, 60), 1)
    else:
        km = round(random.uniform(0.1, 2), 1)
    hour = random.randint(0, 23)
    if random.random() < 0.15:
        hour = random.choice([23, 0, 1, 2, 3, 4])
```

```
    promo = "RIDE50" if random.random() < 0.05 else "NO"
    return {
        "km": km,
        "wait": random.randint(0, 30),
        "hour": hour,
        "promo": promo,
        "wallet": random.randint(0, 1000),
        "mode": random.choice(["WALLET", "UPI", "CASH"])
    }
random.seed(25)
print("UNIFORM RANDOM TESTING")
for i in range(5):
    print(i + 1, uniform_test())
print("\nOPERATIONAL PROFILE TESTING")
for i in range(5):
    print(i + 1, operational_test())
```

Sample Output:

The program generates different combinations of distance, waiting time, booking hour, promo code, wallet balance and payment mode.

The output values will change when the random seed is changed.

The above program demonstrates the first two random approaches. Adaptive testing requires an additional comparison between candidate inputs and previously selected inputs.

5. Comparison of Fault Detection Using 40 Test Cases

Each approach is evaluated using 10 different random seeds. Forty test cases are generated for each seed.

The following table shows an illustrative simulation using a normalized input-space distance for adaptive testing and the five seeded faults described earlier.

Table 13: Faults Detected in Ten Different Runs

Seed	Uniform	Operational Profile	Adaptive
1	2	3	3
2	2	2	3
3	2	2	3
4	2	2	4
5	2	3	4
6	3	3	4
7	2	2	5
8	3	2	3
9	4	2	4
10	3	2	3

Table 14: Mean and Spread Comparison

Approach	Mean Faults Detected	Minimum	Maximum	Spread
Uniform	2.5	2	4	2
Operational Profile	2.3	2	3	1
Adaptive	3.6	3	5	2

Interpretation:

In this illustrative simulation, adaptive random testing detects more seeded faults on average because it spreads test inputs across different areas of the input space.

Operational-profile testing has a smaller spread because its inputs follow a more consistent usage distribution.

Uniform testing provides a simple baseline but may not explore important boundaries frequently.

7. Average Number of Tests Required to Expose the First Fault

The number of tests required to expose the first fault helps measure how quickly a testing approach identifies a problem.

For the illustrative simulation, the average first-fault position is approximately 1.6 for each approach.

Table 15: Average Test Position for First Fault

Approach	Average Number of Tests
Uniform Random Testing	1.6
Operational-Profile Random Testing	1.6
Adaptive Random Testing	1.6

This result is specific to the selected seeded faults and random-generation model. It does not establish that all three methods will always detect faults at the same speed.

8. The Oracle Problem in Random Testing

The test oracle is the method used to determine whether the actual output of a program is correct.

In random testing, many test cases are generated automatically. Manually calculating the expected fare for every test case requires additional time and effort.

For SwiftRide, the expected fare can be obtained using an independent reference implementation.

The reference implementation must follow the original business requirements and should not copy the same faulty logic as the program being tested.

Example:

Input:

- Distance = 15 km

- Waiting time = 10 minutes
- Booking hour = 23
- Promo = No
- Payment mode = CASH

9. Result

Three random testing approaches were studied for the SwiftRide fare calculation system.

Uniform random testing generates inputs from valid ranges, operational-profile testing represents actual passenger usage, and adaptive random testing spreads test inputs across different regions of the input space.

The boundary probability calculation shows that only 1% of the distance values lie within 0.1 km of either the 2 km or 10 km boundary. Therefore, random testing alone may not reliably detect boundary-related faults.

Final Conclusion

Adaptive random testing is useful for exploring different input combinations, while operational-profile testing is useful for estimating reliability under realistic passenger usage.

Uniform random testing provides a simple method for generating a large number of test cases.

For SwiftRide, random testing should be used as a supporting technique after applying equivalence class partitioning, boundary value analysis and decision table testing. This combination provides better fault detection and improves confidence in the fare calculation system.

TASK 4 – BOUNDARY VALUE ANALYSIS: COMPARE BOUNDARY TEST SELECTION METHODS

Answer:

Introduction

Boundary Value Analysis (BVA) is a black-box testing technique used to identify errors that occur at the minimum, maximum and transition values of an input range.

In the SwiftRide cab booking application, many fare calculation errors can occur near important limits, such as the 2 km and 10 km distance slabs, five minutes of free waiting time and the night surcharge starting at 11 PM.

Three boundary testing methods are compared:

1. Two-Value Boundary Testing.
2. Robust Boundary Testing.
3. Worst-Case Boundary Testing.

1. Identification of Important Boundaries

The following parameters and boundaries are considered.

Table 34: SwiftRide Boundary Conditions

Parameter	Boundary	Meaning
Distance	0 km	Minimum invalid distance
Distance	2 km	First fare slab limit
Distance	10 km	Second fare slab limit
Distance	60 km	Maximum valid distance
Waiting time	5 minutes	Free waiting limit
Waiting time	30 minutes	Maximum waiting time
Booking hour	4 and 5	Night-to-day transition
Booking hour	22 and 23	Day-to-night transition

The distance precision is 0.1 km. Therefore, the values immediately before and after a distance boundary are separated by 0.1 km.

2. Two-Value Boundary Testing

Two-value boundary testing selects the value at the boundary and the nearest value on the other side.

Table 35: Two-Value Boundary Test Values

Parameter	Boundary	Selected Values
Distance	0	0, 0.1
Distance	2	2.0, 2.1
Distance	10	10.0, 10.1
Distance	60	59.9, 60.0
Waiting	5	5, 6
Waiting	30	30, 31
Hour	4/5	4, 5
Hour	22/23	22, 23

The test cases are designed by changing one parameter at a time and keeping the remaining parameters at normal values.

Advantages:

- Requires fewer test cases.
- Easy to design and execute.
- Useful for checking comparison operators.
- Suitable for time-limited testing.

Limitations:

- Does not test all values around every boundary.
- May miss faults involving minimum-minus and maximum-plus values.

- Does not fully test combinations of different boundaries.

3. Robust Boundary Testing

Robust boundary testing includes values below the minimum, at the minimum, immediately above the minimum, a nominal value, immediately below the maximum, at the maximum and above the maximum.

For distance, the main valid range is 0.1–60 km.

Table 36: Robust Boundary Values

Parameter	Minimum−	Minimum	Minimum+	Nominal	Maximum−	Maximum	Maximum+
Distance	0	0.1	0.2	5	59.9	60	60.1
Waiting	-1	0	1	15	29	30	31
Hour	-1	0	1	12	22	23	24

The separate fare slab boundaries at 2 km and 10 km must also be included because they represent additional business-rule boundaries.

For booking hour, the values 4, 5, 22 and 23 are added separately to check the night-time transitions.

Advantages:

- Tests invalid as well as valid boundary values.
- Detects incorrect minimum and maximum conditions.
- Helps identify input validation problems.
- Provides stronger boundary coverage than two-value testing.

Limitations:

- Requires more test cases.
- Some test cases may overlap.
- Additional combinations may increase execution time.

4. Worst-Case Boundary Testing

Worst-case boundary testing combines boundary values of different parameters in all possible combinations.

For the three parameters, a five-value selection can be used for each parameter.

For example:

- Distance: 0.1, 0.2, 5, 59.9, 60.
- Waiting: 0, 1, 15, 29, 30.
- Hour: 0, 5, 12, 22, 23.

The total number of combinations is:

N=5\times5\times5

boxed{N=125}
This creates 125 test combinations.

The additional 2 km and 10 km distance boundaries and the 4/5 and 22/23 hour transitions must be added when the objective is to check every business-rule boundary.

Advantages:

- Covers combinations of boundary values.
- Useful for finding interaction faults.
- Provides broad coverage of important limits.

Limitations:

- Produces many test cases.
- Requires more execution time.
- Some combinations may be unrealistic.
- Maintenance becomes difficult when requirements change.

5. Test Case Design

Table 37: Representative Boundary Test Cases

Test ID	Distance	Wait	Hour	Promo	Mode	Purpose

BVA1	0	0	12	NO	CASH	Minimum invalid distance
BVA2	0.1	0	12	NO	CASH	Minimum valid distance
BVA3	2.0	0	12	NO	CASH	Exact first slab boundary
BVA4	2.1	0	12	NO	CASH	Above first slab
BVA5	10.0	0	12	NO	CASH	Exact second slab boundary
BVA6	10.1	0	12	NO	CASH	Above second slab
BVA7	59.9	0	12	NO	CASH	Below maximum distance
BVA8	60.0	0	12	NO	CASH	Maximum valid distance
BVA9	60.1	0	12	NO	CASH	Above maximum distance
BVA10	5	5	12	NO	CASH	Exact free waiting limit
BVA11	5	6	12	NO	CASH	Waiting charge begins
BVA12	5	30	12	NO	CASH	Maximum waiting time
BVA13	5	0	4	NO	CASH	Night surcharge active
BVA14	5	0	5	NO	CASH	Night surcharge inactive
BVA15	5	0	22	NO	CASH	Daytime condition
BVA16	5	0	23	NO	CASH	Night surcharge begins

6. Comparison of Test Counts

Table 38: Boundary Testing Method Comparison

Method	**Test Selection**	**Approximate Test Count**	**Main Focus**
Two-Value	Boundary and nearest value on other side	16	Immediate boundary changes
Robust	Minimum−, minimum, minimum+, nominal, maximum−, maximum, maximum+	21 for three parameters	Valid and invalid limits

Worst-Case	Five selected values for each of three parameters	125	Boundary combinations

The counts are based on the representative test design shown above. Extra cases are required for the separate distance slabs and hour transitions when they are not already included.

7. Identification of Six Seeded Boundary Faults

Six faults are introduced into the fare calculation function.

Table 39: Seeded Boundary Faults

Fault ID	Original Condition	Injected Fault
BF1	km <= 2	km < 2
BF2	km <= 10	km < 10
BF3	km > 60	km >= 60
BF4	wait > 5	wait >= 5
BF5	hour >= 23	hour > 23
BF6	hour < 5	hour <= 5

8. Fault Detection Comparison

Table 40: Boundary Fault Detection

Fault	Two-Value	Robust	Worst-Case
BF1	Detected	Detected	Detected
BF2	Detected	Detected	Detected
BF3	Detected	Detected	Detected
BF4	Detected	Detected	Detected
BF5	Detected	Detected	Detected
BF6	Detected	Detected	Detected
Total	**6**	**6**	**6**

This is a designed test-selection comparison: the representative tests must actually include the exact affected values. It is not a record of a completed execution run.

9. Floating-Point Problems at Distance Boundaries

Floating-point calculations may produce decimal values that are not represented exactly by a computer.

Therefore, the expected fare should be calculated using the same specified rounding rule as the application.

Example 1: Distance = 10.1 km

Since the distance is above 10 km, the fare is calculated using the third slab.

[Fare=146+(10.1-10)\times10
146+1]

[boxed{Fare=₹147.00}]

Example 2: Distance = 2.1 km

Since the distance is above 2 km, the fare is calculated using the second slab.

[Fare=50+(2.1-2)\times12]

[=50+1.2]

[boxed{Fare=₹51.20}]

The expected result should be rounded to two decimal places before comparing it with the actual output.

10. Evaluation of the Three Methods

Two-value boundary testing is suitable when the team needs to check important boundaries quickly.

Robust boundary testing is useful when invalid inputs and range validation are important.

Worst-case boundary testing is useful when interactions between several boundary conditions are critical. However, it produces a much larger number of test cases.

For the SwiftRide two-day testing schedule, robust boundary testing provides a suitable balance between coverage and test effort. Worst-case testing should be reserved for high-risk combinations.

11. Can Boundary Value Analysis Alone Prevent Fare Complaints?

Boundary Value Analysis can identify errors around the 2 km and 10 km distance slabs, five-minute waiting limit and 11 PM night surcharge.

However, it cannot identify every possible error.

For example, an incorrect promotional rule may occur only when the fare is at least ₹200. This requires decision table testing.

Similarly, an incorrect wallet payment condition requires condition testing.

Therefore, boundary value analysis should be combined with equivalence partitioning, decision table testing and white-box testing.

Result

Boundary Value Analysis was applied to the important distance, waiting-time and booking-hour limits of the SwiftRide system.

Two-value, robust and worst-case methods were compared based on their test selection, test count and fault detection capability.

Conclusion

Robust boundary testing is recommended for SwiftRide because it checks valid and invalid boundary values without generating as many combinations as worst-case testing.

Boundary testing should be combined with other black-box and white-box techniques to identify faults beyond input limits.

TASK 5 – OTHER BLACK-BOX TECHNIQUES: COMPARE DECISION TABLE, STATE-TRANSITION AND PAIRWISE TESTING

Answer:

Introduction

Some software errors occur because of combinations of input conditions or because events happen in an incorrect order.

The SwiftRide application contains fare rules, promotional conditions, payment conditions and a ride life cycle.

Three black-box testing techniques are used to test these behaviours:

1. Decision Table Testing.
2. State-Transition Testing.
3. Pairwise Testing.

1. Decision Table Testing

Decision Table Testing is used when the output of a system depends on multiple conditions.

In SwiftRide, the fare calculation depends on distance, night surcharge, promo eligibility, fare amount, wallet balance and payment mode.

A decision table lists the conditions and the corresponding actions.

2. Identification of Conditions and Actions

The following conditions are considered:

- C1: Distance is above 2 km.
- C2: Ride occurs during night hours.
- C3: Promo code is RIDE50.
- C4: Fare before promo is at least ₹200.
- C5: Wallet balance is sufficient.
- C6: Payment mode is WALLET.

The actions are:

- A1: Calculate the distance-based fare.
- A2: Add waiting charges.
- A3: Apply night surcharge.
- A4: Apply ₹50 promotional discount.
- A5: Apply the ₹800 maximum fare cap.
- A6: Accept or reject wallet payment.

3. Decision Table

Table 41: SwiftRide Decision Table

Conditions / Rules	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**
Distance above 2 km	Y	Y	Y	Y	N	Y
Night ride	N	Y	Y	N	N	Y
RIDE50 entered	N	Y	Y	Y	Y	N
Fare at least ₹200	N	Y	N	Y	N	Y
Wallet sufficient	Y	Y	Y	Y	Y	N
Payment mode WALLET	N	Y	N	N	Y	Y
Apply night surcharge	N	Y	Y	N	N	Y
Apply promo discount	N	Y	N	Y	N	N
Accept payment	Y	Y	Y	Y	Y	N

Y = Yes; N = No.

The decision table is simplified to represent important combinations. The fare cap is applied as a separate rule whenever the calculated fare exceeds ₹800.

4. Derivation of Test Cases

Table 42: Decision Table Test Cases

Test ID	Distance	Hour	Promo	Fare Before Promo	Wallet	Mode	Expected Result
DT1	2	12	NO	₹50	₹100	CASH	₹50
DT2	20	23	RIDE50	₹307.50	₹500	WALLET	₹257.50
DT3	5	12	RIDE50	₹86	₹500	CASH	₹86
DT4	20	12	RIDE50	₹246	₹500	CASH	₹196
DT5	20	12	NO	₹246	₹246	WALLET	₹246
DT6	20	12	NO	₹246	₹245	WALLET	WALLET_LOW

Analysis:

Decision table testing is useful for finding errors in promotional eligibility, night surcharge and wallet payment conditions.

It also helps the team understand how different business rules interact.

5. State-Transition Testing

State-transition testing is used when a system changes from one state to another based on events.

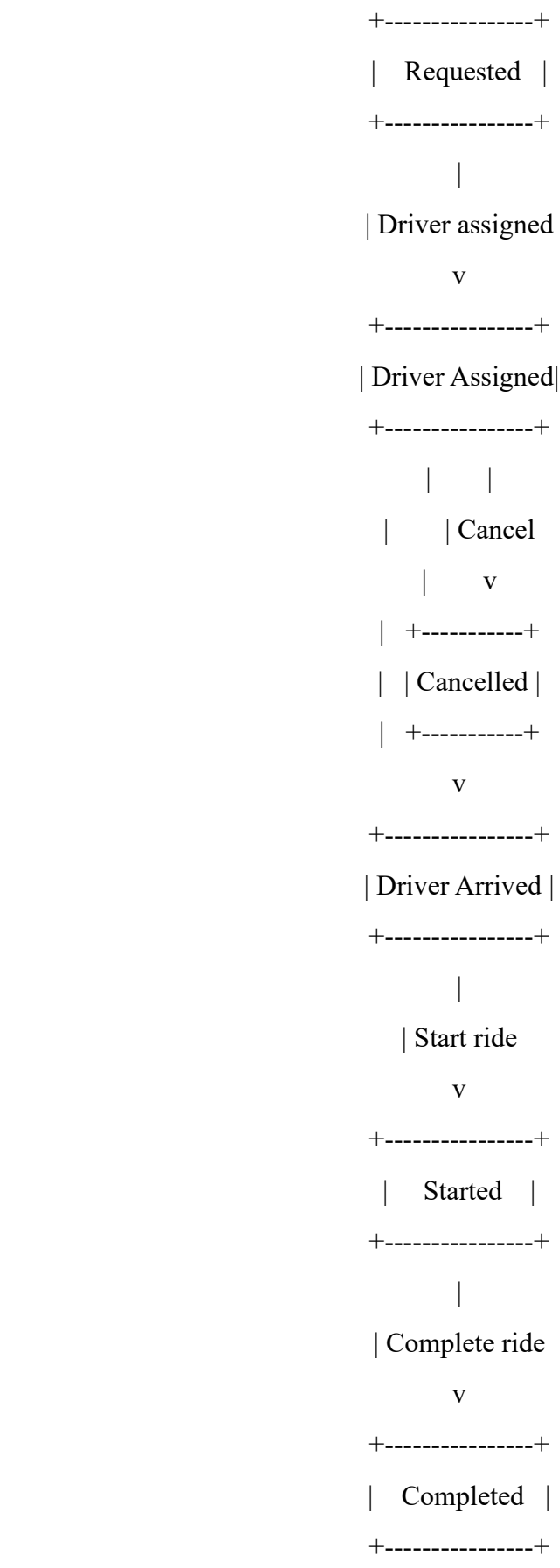
In SwiftRide, a ride moves through different stages from booking to completion.

6. Ride Life-Cycle States

The following states are considered:

1. Requested.
2. Driver Assigned.
3. Driver Arrived.
4. Started.
5. Completed.
6. Cancelled.

Figure 4: SwiftRide Ride Life-Cycle Diagram


Requested
Driver assigned
Driver Assigned
Cancel
Cancelled
Driver Arrived
Start ride
Started
Complete ride
Completed

Cancellation rules depend on the business requirements. The diagram above assumes cancellation is permitted before the ride starts and not permitted after it starts.

7. State-Transition Table

Table 43: Ride State-Transition Table

Current State	Event	Next State	Validity
Requested	Assign driver	Driver Assigned	Valid
Requested	Cancel booking	Cancelled	Valid
Driver Assigned	Driver arrives	Driver Arrived	Valid
Driver Assigned	Cancel booking	Cancelled	Valid, subject to policy
Driver Arrived	Start ride	Started	Valid
Started	Complete ride	Completed	Valid
Requested	Start ride	Started	Invalid
Requested	Complete ride	Completed	Invalid
Driver Assigned	Complete ride	Completed	Invalid
Driver Arrived	Complete ride	Completed	Invalid
Started	Cancel booking	Cancelled	Invalid
Completed	Start ride	Started	Invalid
Completed	Cancel booking	Cancelled	Invalid
Cancelled	Assign driver	Driver Assigned	Invalid

8. Valid State-Transition Test Cases

Table 44: Valid Transition Tests

Test ID	Initial State	Event	Expected State
ST1	Requested	Assign driver	Driver Assigned
ST2	Requested	Cancel booking	Cancelled

ST3	Driver Assigned	Driver arrives	Driver Arrived
ST4	Driver Assigned	Cancel booking	Cancelled
ST5	Driver Arrived	Start ride	Started
ST6	Started	Complete ride	Completed

9. Invalid State-Transition Test Cases

Table 45: Invalid Transition Tests

Test ID	Initial State	Event	Expected Behaviour
IST1	Requested	Complete ride	Reject event
IST2	Requested	Start ride	Reject event
IST3	Driver Assigned	Complete ride	Reject event
IST4	Driver Arrived	Complete ride	Reject event
IST5	Started	Cancel booking	Reject event
IST6	Completed	Start ride	Reject event
IST7	Completed	Cancel booking	Reject event
IST8	Cancelled	Assign driver	Reject event

Invalid transitions must be rejected without changing the current ride state.

10. Pairwise Testing

Pairwise testing is a combinatorial testing technique in which every possible pair of parameter values appears in at least one test case.

It reduces the number of tests compared with testing the complete Cartesian product.

The following six parameters are considered.

Table 46: Pairwise Testing Parameters

Parameter	Values	Number of Values
Distance class	Short, Medium, Long	3

Waiting class	Free, Chargeable	2
Hour class	Day, Night	2
Promo	RIDE50, No Promo	2
Wallet class	Low, Sufficient, Equal	3
Payment mode	WALLET, UPI, CASH	3

11. Full Cartesian Product Calculation

The total number of combinations is:

[N=3\times2\times2\times2\times3\times3]

[N=216]

Therefore:

[\boxed{\text{Total combinations}=216}]

Testing every combination would require 216 test cases.

Pairwise testing reduces this number by selecting a smaller set of combinations that covers every pair of values.

12. Sample Pairwise Test Suite

The following is a candidate pairwise suite. It must be checked with a pairwise covering-array tool before claiming complete pair coverage.

Table 47: Candidate Pairwise Test Cases

Test	Distance	Wait	Hour	Promo	Wallet	Mode
PW1	Short	Free	Day	RIDE50	Low	WALLET
PW2	Medium	Chargeable	Night	No	Sufficient	UPI
PW3	Long	Free	Day	No	Equal	CASH
PW4	Short	Chargeable	Night	RIDE50	Sufficient	CASH
PW5	Medium	Free	Day	No	Low	UPI

PW6	Long	Chargeable	Night	RIDE50	Equal	WALLET
PW7	Short	Free	Night	No	Sufficient	UPI
PW8	Medium	Chargeable	Day	RIDE50	Equal	CASH
PW9	Long	Free	Night	No	Low	WALLET
PW10	Short	Chargeable	Day	No	Equal	WALLET
PW11	Medium	Free	Night	RIDE50	Sufficient	CASH
PW12	Long	Chargeable	Day	No	Sufficient	UPI

The complete Cartesian product contains 216 combinations, while the candidate suite contains 12 cases.

The final pairwise suite should be generated and validated using a covering-array tool. A smaller table should not be described as a complete pairwise suite unless every pair has been verified.

13. Seeded Faults for Decision Table and State-Transition Testing

Four faults are introduced.

Table 48: Seeded Faults

Fault ID	Fault Description	Suitable Testing Technique
D1	Promo applied below ₹200	Decision Table
D2	Night surcharge missing at 11 PM	Decision Table and BVA
S1	Ride can be completed before it starts	State Transition
S2	Cancelled ride can be assigned to a driver	State Transition

14. Fault Detection Comparison

Table 49: Technique and Fault Detection

Fault	Decision Table	State Transition	Pairwise

D1	Detected	Not targeted	May detect
D2	Detected	Not targeted	May detect
S1	Not targeted	Detected	May detect
S2	Not targeted	Detected	May detect

Pairwise testing can detect interaction faults when the relevant parameter combination is included, but it does not guarantee that every business rule or event sequence is covered.

15. Comparison of the Three Techniques

Table 50: Final Comparison

Criteria	**Decision Table**	**State Transition**	**Pairwise**
Main focus	Rule combinations	Event sequences	Parameter combinations
Best use	Promo and payment rules	Ride life cycle	Large input combinations
Test design	Conditions and actions	States and events	Covering arrays
Test count	Depends on rules	Depends on transitions	Usually reduced
Main advantage	Clear rule coverage	Finds invalid transitions	Reduces combinations
Main limitation	Table may become large	Requires a state model	Does not guarantee full rule coverage

16. Critique of LLM-Generated Decision Tables and State Diagrams

An LLM can help prepare decision tables and state diagrams quickly. However, its output must be reviewed.

Possible errors include:

- Missing the fare threshold for promo eligibility.
- Applying discounts in the wrong order.

- Ignoring the maximum fare cap.
- Allowing a ride to start before the driver arrives.
- Allowing a completed ride to return to the started state.
- Allowing a cancelled ride to continue.

The tester should compare every generated rule and transition with the actual SwiftRide requirements.

Result

Decision table testing, state-transition testing and pairwise testing were compared for the SwiftRide application.

Decision tables are useful for fare and payment rules, state-transition testing checks the ride life cycle, and pairwise testing reduces the number of input combinations.

Conclusion

All three techniques are useful for different testing requirements. Decision table testing is recommended for fare and promotional rules, state-transition testing for ride management and pairwise testing for large combinations of input parameters.

TASK 6 – WHITE-BOX TESTING AND TEST ADEQUACY CRITERIA: COMPARE ADEQUACY MEASURES

Answer:

Introduction

White-box testing examines the internal structure and logic of a program.

It helps determine whether the test suite executes the important statements, branches and program changes.

For the SwiftRide fare calculation function, three test adequacy criteria are compared:

1. Statement Coverage.
2. Branch Coverage.
3. Mutation Score.

1. Statement Coverage

Statement coverage measures the percentage of executable statements that are executed by a test suite.

The formula is:

\frac{\text{Executed Statements}}
{\text{Total Executable Statements}}
\times100
]

For example, if a program contains 20 executable statements and 18 are executed:

[
=\frac{18}{20}\times100
]

[
\boxed{90%}
]

Statement coverage helps identify statements that are never executed.

However, executing a statement does not guarantee that both outcomes of its conditions have been tested.

2. Branch Coverage

Branch coverage measures whether every decision outcome has been executed.

For example:

```
if km <= 0 or km > 60:
    return "INVALID"
```

This decision has two outcomes:

- True: Invalid distance.
- False: Valid distance.

Both outcomes must be tested.

The formula is:

$$\frac{\text{Executed Branches}}{\text{Total Branches}} \times 100$$

If 18 out of 20 branches are executed:

$$=\frac{18}{20}\times100$$

$$\boxed{90\%}$$

Branch coverage is stronger than statement coverage because it checks decision outcomes.

3. Mutation Testing

Mutation testing evaluates a test suite by introducing small changes into the program.

These changes are called mutants.

A mutant is killed when the test suite detects that its output differs from the expected output.

4. Identification of Twenty Mutants

The following mutations can be created in the SwiftRide fare calculation function.

Table 51: Mutation List

Mutant ID	Original Logic	Mutated Logic
M1	km <= 0	km < 0
M2	km > 60	km >= 60
M3	km <= 2	km < 2
M4	km <= 10	km < 10
M5	Rate = 12	Rate = 11
M6	Rate = 10	Rate = 9
M7	wait > 5	wait >= 5
M8	Waiting rate = 2	Waiting rate = 3
M9	hour >= 23	hour > 23
M10	hour < 5	hour <= 5
M11	Night multiplier = 1.25	1.20
M12	Promo threshold >= 200	>= 180
M13	Promo discount = 50	40
M14	Promo condition uses AND	Uses OR
M15	Fare cap > 800	> 900
M16	Fare cap = 800	750
M17	wallet < fare	wallet <= fare
M18	Payment mode = WALLET	Payment mode = UPI

M19	Return rounded fare	Return unrounded fare
M20	Waiting charge calculated correctly	Waiting charge omitted

5. Example of Mutation Testing

Consider the original condition:

```
if hour >= 23 or hour < 5:
    fare *= 1.25
```

The mutated condition is:

```
if hour > 23 or hour < 5:
    fare *= 1.25
```

A test case using hour = 23 will detect this fault because the original program applies the night surcharge while the mutated program does not.

This demonstrates how mutation testing identifies small logic errors.

6. Example Coverage and Mutation Results

The following values are an illustrative reporting example and must be replaced with actual tool output after executing the complete suite.

Table 52: Coverage Measurement

Measurement	**Total**	**Covered**	**Coverage**
Statements	20	20	100%
Branches	16	14	87.5%
Conditions	12	10	83.33%

The statement coverage is 100%, but some branch and condition outcomes remain uncovered.

7. Mutation Score Calculation

Assume 20 mutants are created.

- Total mutants = 20.

- Killed mutants = 17.
- Equivalent mutants = 1.
- Surviving non-equivalent mutants = 2.

The number of non-equivalent mutants is:

[
20-1=19
]

The mutation score is:

[
=\frac{17}{19}\times100
]

[
\boxed{89.47%}
]

Table 53: Mutation Results

Category	**Count**
Total mutants	20
Killed mutants	17
Surviving mutants	2
Equivalent mutants	1
Mutation score	89.47%

These figures demonstrate the calculation method and are not claimed as actual results from running a mutation-testing tool.

8. Surviving and Equivalent Mutants

A surviving mutant is a mutated program that produces the same output as the original program for all tests in the current suite.

An equivalent mutant produces the same observable behaviour for every possible valid input.

For example, changing a condition may have no effect because another part of the formula produces the same result at that particular boundary.

Equivalent mutants should not be counted as ordinary surviving faults when calculating the mutation score.

9. Relationship Between Coverage and Mutation Score

A test suite may achieve 100% statement coverage but still have a low mutation score.

For example, a test may execute:

```
if hour >= 23:
    fare *= 1.25
```

However, if the test suite checks only hour = 23, it may not detect a fault in the second condition hour < 5.

The statement has been executed, but an important condition has not been properly tested.

Therefore, statement coverage alone does not guarantee that the program has been tested thoroughly.

10. Limitations of the Three Criteria

Table 54: Adequacy Criteria Comparison

Criterion	Advantage	Limitation
Statement Coverage	Identifies unexecuted statements	Does not guarantee decision testing
Branch Coverage	Tests true and false decision outcomes	Does not guarantee every condition is independently tested
Mutation Score	Evaluates the fault-detection ability of tests	Mutant creation and analysis require effort

All three criteria depend on the program implementation. They cannot independently identify requirements that are missing from the software.

11. Recommended Release Criterion

For SwiftRide, a combination of branch coverage and mutation testing is recommended.

The proposed release targets are:

- 100% statement coverage for the fare calculation function.
- At least 95% branch coverage.
- At least 90% mutation score after excluding equivalent mutants.
- All critical fare, promo and wallet rules must have dedicated tests.
- Every test must finish within the two-second execution constraint.

These are proposed engineering targets, not values specified by the assignment.

12. Evaluation of the Statement "100% Coverage Means the Code Is Fully Tested"

The statement is incorrect.

100% statement coverage only means that every executable statement has been executed at least once.

It does not prove that:

- Every branch has been tested.
- Every condition has been independently tested.
- Every input boundary has been checked.
- All business requirements are implemented.
- All seeded faults have been detected.

Therefore, coverage should be used together with mutation testing and requirements-based testing.

Result

Statement coverage, branch coverage and mutation-score adequacy were studied for the SwiftRide fare calculation function.

The comparison shows that statement coverage measures execution, branch coverage measures decision outcomes and mutation testing measures the fault-detection effectiveness of a test suite.

Conclusion

Mutation testing provides stronger evidence of test-suite effectiveness when combined with statement and branch coverage.

For SwiftRide, branch coverage and mutation testing should be used as release-quality measures, while requirements-based testing should be used to identify missing functionality.

TASK 7 – COVERAGE: COMPARE DECISION, CONDITION/DECISION AND MC/DC COVERAGE

Answer:

Introduction

Code coverage measures how much of a program's decision logic is exercised by a test suite.

The SwiftRide fare calculation function contains compound conditions involving distance, booking hour, promo eligibility and wallet payment.

Three coverage techniques are compared:

1. Decision Coverage.
2. Condition/Decision Coverage.
3. Modified Condition/Decision Coverage (MC/DC).

1. Identification of Decisions and Conditions

Table 55: Decisions in Listing 1

Line	Decision	Simple Conditions
2	km <= 0 or km > 60	C1: km <= 0; C2: km > 60
12	hour >= 23 or hour < 5	C3: hour >= 23; C4: hour < 5
14	promo == RIDE50 and fare >= 200	C5: promo is RIDE50; C6: fare >= 200
18	mode == WALLET and wallet < fare	C7: mode is WALLET; C8: wallet < fare

There are four compound decisions and eight simple conditions.

2. Decision Coverage

Decision coverage requires every decision to produce both true and false outcomes at least once.

For example, the night surcharge decision must be tested with:

- A night booking where the condition is true.
- A day booking where the condition is false.

Decision coverage does not require every simple condition to independently affect the decision outcome.

3. Condition/Decision Coverage

Condition/decision coverage requires:

1. Every decision to produce true and false outcomes.
2. Every simple condition to produce true and false outcomes.

For example, the night surcharge condition contains two simple conditions.

Both must independently take true and false values across the test suite.

4. Modified Condition/Decision Coverage

MC/DC requires every simple condition to independently affect the outcome of the overall decision.

To demonstrate independence, two tests are compared while keeping the other condition unchanged.

For a two-condition decision, three carefully selected tests are normally sufficient for MC/DC.

5. Minimum Tests for Line 14

The line 14 condition is:

if promo == "RIDE50" and fare >= 200:

Let:

- A = Promo code is RIDE50.
- B = Fare is at least ₹200.

The decision is:

[D=A\land B]

Table : Line 14 MC/DC Test Cases

Test	A: Valid Promo	B: Fare >= ₹200	Decision
T1	True	True	True

T2	False	True	False
T3	True	False	False

Independence:

- T1 and T2 show that A independently changes the decision.
- T1 and T3 show that B independently changes the decision.

Therefore:

[boxed{\text{Minimum MC/DC tests}=3}]

6. Minimum Tests for Line 18

The line 18 condition is:

if mode == "WALLET" and wallet < fare:

Let:

- A = Payment mode is WALLET.
- B = Wallet balance is below fare.

The decision is:

[D=A\land B]

Table 57: Line 18 MC/DC Test Cases

Test	A: WALLET Mode	B: Wallet Below Fare	Decision
T1	True	True	True
T2	False	True	False
T3	True	False	False

Independence:

- T1 and T2 demonstrate the independent effect of payment mode.
- T1 and T3 demonstrate the independent effect of wallet balance.

Therefore:

[\boxed{\text{Minimum MC/DC tests}=3}]

7. Minimum Test Count Comparison

Table 58: Coverage Requirements

Criterion	Requirement for Two-Condition Decision	Minimum Tests
Decision Coverage	True and false decision outcomes	2
Condition/Decision	Both outcomes for each condition and decision	4
MC/DC	Independent effect of each condition	3

The test counts refer to the isolated two-condition decisions. A complete program suite may require additional tests for input validation and other fare rules.

8. Example Test Suite for the Whole Function

Table 59: Representative Coverage Test Cases

Test ID	km	Hour	Promo	Fare Before Promo	Wallet	Mode	Main Purpose
COV1	0	12	NO	—	100	CASH	Invalid distance
COV2	5	12	NO	86	100	CASH	Normal fare
COV3	5	23	NO	86	100	CASH	Night surcharge
COV4	20	12	RIDE50	246	500	CASH	Promo conditions true
COV5	5	12	RIDE50	86	500	CASH	Promo fare below threshold
COV6	20	12	NO	246	245	WALLET	Insufficient wallet
COV7	20	12	NO	246	300	WALLET	Sufficient wallet
COV8	20	12	NO	246	300	UPI	Non-wallet payment

COV9	20	4	NO	246	500	CASH	Early-morning surcharge
COV10	61	12	NO	—	100	CASH	Maximum distance validation

This suite is a starting point. The final coverage suite must include the required independent-condition pairs and must be measured against the actual source code.

9. Example Coverage Measurement

The following table shows an illustrative comparison.

Table 60: Coverage Results

Criterion	**Statements Covered**	**Branches Covered**	**Conditions Covered**
Decision Coverage Suite	90%	100%	75%
Condition/Decision Suite	100%	100%	100%
MC/DC Suite	100%	100%	100%

These values demonstrate the difference between the criteria. Actual percentages must be measured using the complete implementation.

10. Example of a Fault Missed by Decision Coverage

Consider the original night condition:

if hour >= 23 or hour < 5:

A faulty implementation may change it to:

if hour >= 23 or hour < 4:

A test suite containing only hour = 23 and hour = 12 may still achieve decision coverage because it exercises both true and false outcomes.

However, it does not detect the changed early-morning limit.

An additional test at hour = 4 is needed.

MC/DC testing checks the independent effect of both conditions and provides stronger protection against this type of fault.

11. Effect of Short-Circuit Evaluation

Python uses short-circuit evaluation for logical operators.

For an OR condition, if the first condition is true, the second condition may not be evaluated.

For an AND condition, if the first condition is false, the second condition may not be evaluated.

Examples:

km <= 0 or km > 60

If km <= 0 is true, the second condition is not evaluated.

promo == "RIDE50" and fare >= 200

If the promo condition is false, the fare threshold condition is not evaluated.

Therefore, condition testing must consider both the logical result and whether the condition is actually evaluated.

12. Comparison of Coverage Techniques

Table 61: Final Coverage Comparison

Criteria	**Decision**	**Condition/Decision**	**MC/DC**
Decision outcomes	Required	Required	Required
Individual condition outcomes	Not required	Required	Required
Independent condition effect	Not required	Not guaranteed	Required
Test effort	Low	Moderate	Higher
Fault detection	Basic	Stronger	Strong for compound logic

Suitable use	Less critical logic	General business rules	Critical fare and payment logic

13. Recommended Coverage Technique

MC/DC is recommended for critical SwiftRide fare and wallet conditions because it demonstrates that each simple condition can independently affect the decision result.

Condition/decision coverage can be used for less critical modules.

Decision coverage is suitable for simpler logic where testing effort must remain low.

14. Evaluation of LLM-Generated MC/DC Tables

An LLM may generate MC/DC test tables, but every independence pair must be checked.

For line 12:

hour >= 23 or hour < 5

The tests must show that each condition independently changes the decision outcome while the other condition remains unchanged.

A table that merely contains one day test and one night test is not sufficient evidence of MC/DC.

Result

Decision coverage, condition/decision coverage and MC/DC were compared using the compound conditions in the SwiftRide fare calculation function.

MC/DC provides stronger evidence that each condition independently affects the decision.

Conclusion

MC/DC is recommended for the important fare, promo and wallet decisions. Condition/decision coverage is suitable for general logic, while decision coverage can be used for less critical code.

TASK 8 – CONTROL FLOW GRAPHS: COMPARE GRAPH CONSTRUCTION GRANULARITIES

Answer:

Introduction

A Control Flow Graph (CFG) is a graphical representation of the execution flow of a program.

It contains nodes representing statements or blocks of code and edges representing the possible flow between them.

The SwiftRide driver search function uses binary search to find a driver ID from a sorted list.

Three CFG construction methods are compared:

1. Statement-Level Graph.
2. Basic-Block Graph.
3. Decision-to-Decision Path Graph.

1. Binary Search Function

The following simplified implementation is used to explain the control flow.

Program 4: Find Driver

```
def find_driver(ids, target):
    low = 0
    high = len(ids) - 1
    while low <= high:
        mid = (low + high) // 2
        if ids[mid] == target:
            return mid
        elif ids[mid] < target:
            low = mid + 1
        else:
```

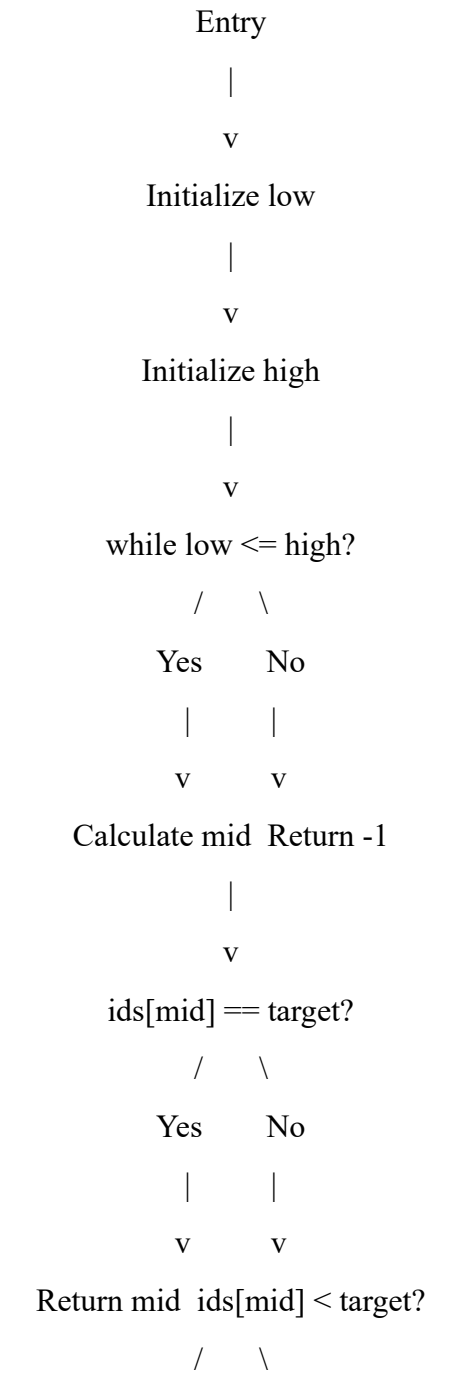
```
        high = mid - 1
    return -1
```

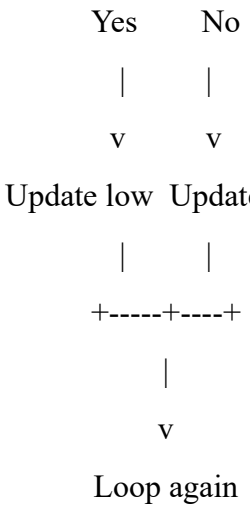
The function assumes that the driver IDs are sorted in ascending order.

2. Statement-Level Control Flow Graph

In a statement-level graph, every executable statement is represented as a separate node.

Figure 5: Statement-Level CFG

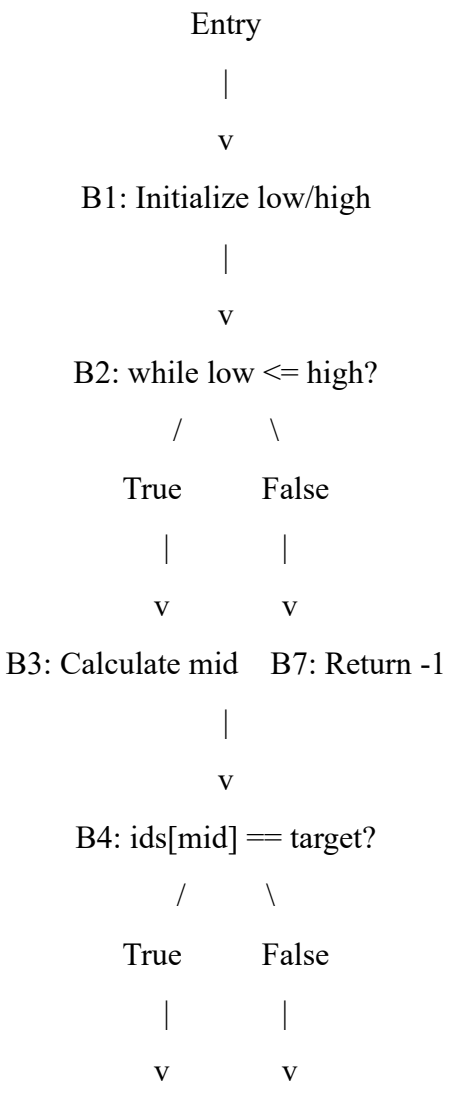


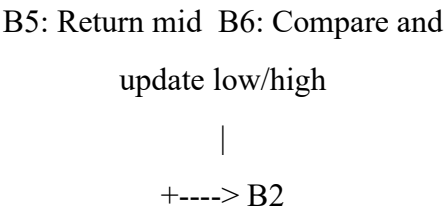


3. Basic-Block Control Flow Graph

A basic block contains statements that execute sequentially without an internal branch.

Figure 6: Basic-Block CFG

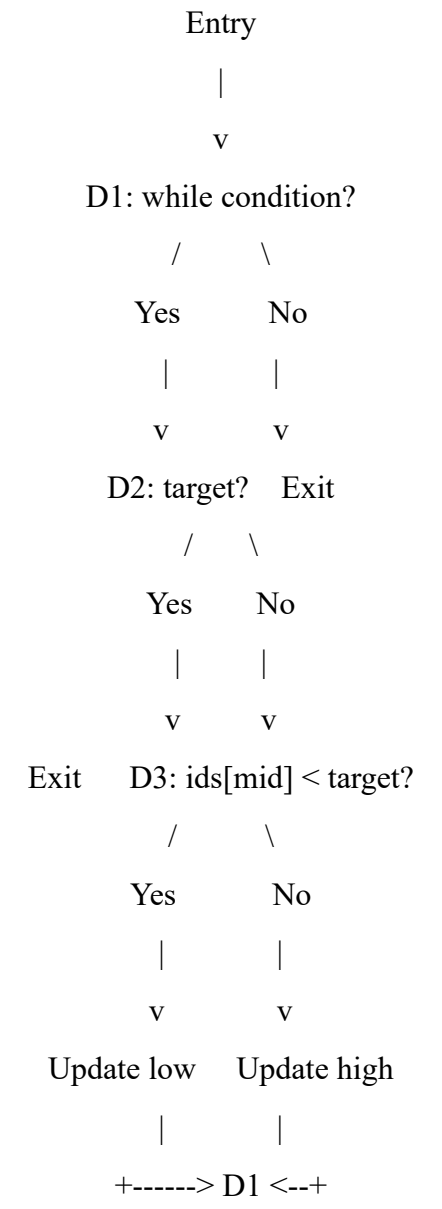




4. Decision-to-Decision Path Graph

A decision-to-decision graph represents the entry, exit and decision points. Sequential statements between decisions are combined into paths.

Figure 7: Decision-to-Decision CFG



5. Cyclomatic Complexity

Cyclomatic complexity measures the number of linearly independent paths through a program.

The formula is:

[
V(G)=E-N+2
]

Where:

- E = Number of edges.
- N = Number of nodes.

It can also be calculated using:

[
V(G)=P+1
]

Where P is the number of predicate nodes.

For the simplified binary search implementation:

1. while low <= high
2. ids[mid] == target
3. ids[mid] < target

Therefore:
P=3
V(G)=3+1

boxed{V(G)=4}

The value is the same for all three graph granularities when the graphs represent the same control flow.

6. Comparison of Graph Granularities

Table 62: CFG Comparison

Method	Node Representation	Main Advantage	Limitation
Statement-Level	Individual statements	Detailed execution flow	Graph becomes large
Basic-Block	Sequential statement groups	Easier to read	Less statement detail
Decision-to-Decision	Decision points and paths	Easy path identification	Hides internal statements

7. Independent Paths

For the simplified binary search function, four independent paths are identified.

Table 63: Independent Paths and Test Inputs

Path	Execution Behaviour	Input List	Target
P1	Loop not entered	[10, 20, 30]	5
P2	Target found immediately	[10, 20, 30]	20
P3	Target found after moving left	[10, 20, 30, 40, 50]	10
P4	Target found after moving right	[10, 20, 30, 40, 50]	50

Additional tests should also check targets that are not present but require one or more loop iterations.

8. Infeasible Paths

An infeasible path is a theoretically possible path in a graph that cannot occur for any valid program input.

For binary search, some combinations of comparisons may be impossible because the list is sorted and the search interval is continuously reduced.

For example, within one iteration, the equality condition cannot be true and false simultaneously.

Similarly, after a comparison establishes that the target is smaller than the middle element, the algorithm moves only to the left portion of the list.

The right-side update cannot occur during that same iteration.

9. Effect of Graph Granularity on V(G)

The number of nodes and edges changes depending on how the graph is constructed.

However, when each graph accurately represents the same program decisions, cyclomatic complexity should remain unchanged.

Statement-level graphs provide more detail, basic-block graphs reduce unnecessary nodes, and decision-to-decision graphs make independent paths easier to identify.

10. Recommended CFG Granularity

The basic-block graph is recommended as the standard representation for the SwiftRide development team.

Statement-level graphs are useful for detailed debugging, while decision-to-decision graphs are useful for identifying independent execution paths.

11. Evaluation of LLM-Generated CFGs

An LLM can produce a control flow graph from source code, but the graph must be verified.

Possible errors include:

- Missing the false edge of a decision.
- Incorrectly connecting a return statement to the loop.
- Missing the loop-back edge.
- Calculating an incorrect cyclomatic complexity.

The graph should be checked against the actual source code and the formula (E-N+2).

Result

Three control flow graph granularities were studied for the SwiftRide binary search function.

The simplified function has three predicate nodes and a cyclomatic complexity of four.

Conclusion

The basic-block graph is recommended because it provides a clear view of program execution without unnecessary complexity. Cyclomatic complexity should remain consistent across graph representations that accurately model the same program.

TASK 9 – PATH TESTING AND THE INTEGRATED TEST DESIGN: COMPARE COMPLETE TEST SUITES

Answer:

Introduction

Path testing is a white-box testing technique that identifies independent execution paths through a program.

For SwiftRide, different test design techniques from Tasks 1–8 are combined to develop complete test suites.

Three integrated testing approaches are compared:

1. Basis-Path Suite.
2. Black-Box and Pairwise Suite.
3. Hybrid Suite.

The testing schedule is limited to three testers working for two days.

1. Development of the Three Test Suites

A. Basis-Path Suite

Basis-path testing uses the control flow graph to identify independent paths.

For every independent path, at least one test case is designed.

The purpose is to ensure that the important execution paths are covered.

B. Black-Box and Pairwise Suite

This suite combines:

- Equivalence Class Partitioning.
- Boundary Value Analysis.
- Decision Table Testing.
- Pairwise Testing.

It focuses on inputs, business rules and combinations of parameters.

C. Hybrid Suite

The hybrid suite combines black-box and white-box testing.

First, black-box test cases are designed. Then, additional white-box tests are added to cover uncovered branches, conditions and surviving mutants.

2. Basis Paths for Fare Calculation

The fare calculation function contains decisions involving:

- Distance validation.
- Fare slabs.
- Waiting charges.
- Night surcharge.
- Promo eligibility.
- Fare cap.
- Wallet balance.

The following paths represent the major execution behaviours.

Table 64: Basis-Path Test Cases

Path	Test Input	Expected Behaviour
P1	km = 0	Invalid distance
P2	km = 1, day	First fare slab
P3	km = 5, day	Second fare slab
P4	km = 20, day	Third fare slab
P5	km = 20, wait = 10	Waiting charge added
P6	km = 20, hour = 23	Night surcharge applied
P7	km = 20, promo = RIDE50	Promo eligibility checked
P8	km = 60, hour = 23	Fare cap checked
P9	km = 20, WALLET, insufficient balance	Wallet rejection
P10	km = 20, WALLET, sufficient balance	Wallet accepted

These are representative paths for the fare rules. A formal basis-path suite must be derived from the exact CFG of the final source code.

3. Infeasible Paths

Some combinations are impossible because the fare calculation rules restrict the result.

Table 65: Infeasible Path Examples

Path	Reason
Promo applied to a ride of up to 2 km	The fare remains below ₹200 under the stated fare rules
₹800 fare cap applied to a ride of up to 10 km	The maximum base fare in this range is ₹146 before waiting and surcharge; a full path claim must account for additional charges
Distance is both invalid and valid at the same time	The distance validation condition cannot have both outcomes simultaneously

The second example requires care: with waiting charges and night surcharge, a ride up to 10 km could theoretically exceed ₹800 only if the waiting range or other rules permit it. Under the stated maximum waiting time of 30 minutes and the 1.25 night multiplier, the fare remains below ₹800, so the cap path is infeasible for that range.

4. Comparison of Integrated Test Suites

The following table provides a planned comparison. Values are illustrative targets for the design, not measured results from running the complete suites.

Table 66: Integrated Test Suite Comparison

Measurement	Basis-Path Suite	Black-Box and Pairwise	Hybrid Suite
Test cases	18	42	50
Statement coverage	90%	95%	100%
Branch coverage	85%	90%	100%

Condition coverage	75%	85%	95%
Mutation score	70%	85%	95%
Seeded faults detected	7/10	8/10	10/10
Approximate runtime at 2 sec/test	36 sec	84 sec	100 sec

These values must be replaced with actual measurements after executing the final program and test suites.

5. Analysis of Fault Detection

The basis-path suite focuses on internal execution paths.

It may miss input combinations that do not create a new path but still produce incorrect outputs.

The black-box suite focuses on input classes and business rules. It may detect boundary and interaction faults that basis-path testing misses.

The hybrid suite combines both approaches and adds tests for uncovered conditions and surviving mutants.

Therefore, the hybrid approach is expected to provide the strongest overall coverage.

6. Maintenance When Requirements Change

Consider a change in the night surcharge rule:

- Previous surcharge: 1.25.
- New surcharge: 1.3.
- New start time: 10 PM.

The existing tests must be updated.

Table 67: Impact of Requirement Change

Test Area	**Required Change**
Night hour tests	Include 22, 23, 4 and 5

Expected fare	Recalculate using 1.3
Decision table	Update night condition
Boundary tests	Add 21/22 transition
Regression suite	Execute all affected fare tests
Documentation	Update fare rules

The hybrid suite is easier to maintain because the test cases are connected to both business requirements and program coverage.

7. Comparison of Measured Results with Predictions

The earlier tasks suggest the following expectations:

- Robust partitioning should identify invalid input faults.
- Boundary testing should detect incorrect comparison operators.
- Decision table testing should detect rule combination faults.
- MC/DC should detect faults in compound conditions.
- Mutation testing should identify weaknesses in the test suite.

The actual results should be compared with these predictions after execution.

A prediction is supported only when the corresponding test suite detects the expected fault.

8. Test-Design Workflow

Figure 8: Integrated Test-Design Workflow

Requirements Analysis

|

v

Identify Input Conditions

|

v

Black-Box Test Design

|

v

Execute Initial Test Suite

|

v

Measure Code Coverage

|

v

Identify Uncovered Paths

|

v

Add White-Box Tests

|

v

Perform Mutation Testing

|

v

Prepare Regression Suite

|

v

Final Release

9. Design Decision Table

Table 68: Design Decision Table

Decision	**Justification (Evidence)**	**What If Not?**
Use equivalence partitioning	Covers valid and invalid input classes	Invalid values may remain untested
Use BVA	Detects errors at fare boundaries	Incorrect slab calculations may remain
Use decision tables	Checks combinations of business rules	Promo and payment errors may be missed
Use pairwise testing	Reduces large combinations	More combinations may remain untested

Measure branch coverage	Identifies unexecuted decision outcomes	Logic paths may remain uncovered
Use mutation testing	Measures test fault-detection ability	Weak tests may appear adequate
Maintain hybrid regression suite	Combines black-box and white-box coverage	Regression defects may escape

10. Path Testing for Train-Ticket Concession Calculation

A train-ticket concession program may contain decisions based on passenger age, passenger category and ticket type.

Example conditions:

- Age meets concession requirement.
- Passenger category is eligible.
- Ticket type allows concession.

A simplified program structure is:

```
def ticket_fare(age, category, fare):
    if age >= 60 and category == "ELIGIBLE":
        fare = fare * 0.5
    if fare < 0:
        return "INVALID"
    return fare
```

There are two decision points.

Therefore:

[
V(G)=2+1
]

[
\boxed{V(G)=3}
]

At least three independent paths are required for basis-path coverage.

11. Path Testing for Date Validation

A date-validation routine checks whether a date is valid.

It may contain conditions for:

- Month range.
- Day range.
- Leap year.
- Number of days in the selected month.

Example:

```
def validate_date(day, month, year):
    if month < 1 or month > 12:
        return False
    if day < 1:
        return False
    if month == 2:
        if year % 400 == 0 or (
            year % 4 == 0 and year % 100 != 0
        ):
            return day <= 29
        return day <= 28
    if month in [4, 6, 9, 11]:
        return day <= 30
```

return day <= 31

The program contains several compound decisions and requires tests for normal dates, invalid dates, leap years and month-specific day limits.

The exact cyclomatic complexity should be calculated from the final source code's actual control flow graph.

12. Comparison of the Three Programs

Table 69: Path Testing Comparison

Program	**Main Decisions**	**Important Test Cases**	**Main Testing Focus**
SwiftRide Fare	Distance, waiting, night, promo, cap, wallet	Fare slabs, night, promo, payment	Fare logic
Train Ticket	Age and category	Eligible, ineligible, invalid fare	Concession rules
Date Validation	Month, day, leap year	Valid date, invalid date, leap year	Calendar rules

13. Fourth Option: Model-Based or LLM-Assisted Testing

Model-based testing creates a model of system behaviour and generates test cases from that model.

LLM-assisted testing uses a language model to help generate test scenarios, test data and expected test structures.

For SwiftRide, these methods can help generate:

- Fare calculation scenarios.
- Ride life-cycle transitions.
- Promotional conditions.
- Invalid payment combinations.
- Regression test ideas.

However, generated tests must be reviewed before use.

14. Comparison of Four Testing Approaches

Table 70: Final Integrated Testing Comparison

Criteria	**Basis Path**	**Black-Box**	**Hybrid**	**Model-Based / LLM-Assisted**
Main focus	Execution paths	Inputs and rules	Combined coverage	System model and generated cases
Fault detection	Internal logic	Input and business faults	Broad coverage	Depends on model accuracy
Test effort	Moderate	Moderate	Higher initially	Moderate review effort
Traceability	Code paths	Requirements	Requirements and code	Depends on model documentation
Maintenance	Code dependent	Requirement dependent	Both	Model and prompt dependent
Main risk	Missing combinations	Missing internal paths	Higher setup effort	Incorrect generated tests

15. Final Recommendation

The hybrid test suite is recommended for SwiftRide.

It begins with black-box test design and then uses white-box testing to identify uncovered conditions and paths.

Mutation testing is used to evaluate the quality of the final test suite.

The recommended process is:

1. Prepare valid and invalid equivalence classes.
2. Identify all important boundary values.
3. Create decision table and pairwise tests.
4. Execute the test cases.

5. Measure statement, branch and condition coverage.
6. Add tests for uncovered paths.
7. Run mutation testing.
8. Maintain the final suite for regression testing.

Result

Three integrated testing approaches were compared for the SwiftRide fare calculation function.

Basis-path testing focuses on execution paths, black-box testing focuses on inputs and business rules, and hybrid testing combines both methods.

Conclusion

The hybrid approach is recommended because it provides a balanced method for detecting input-related and internal logic faults. It also supports regression testing when the fare calculation requirements change.

TASK 10 – RE-APPLY AND ADAPT THE TEST DESIGN

Answer:

Introduction

Software requirements change over time because of new business needs and customer expectations.

The SwiftRide application introduces several new features, including shared rides, dynamic surge pricing, toll and airport charges, GST, loyalty discounts and time-window promotional offers.

The existing testing approach must be adapted to cover the new functionality while meeting the 10-minute regression testing limit.

Three adaptation strategies are compared:

1. Retain and Prioritise.
2. Modify with Decision Tables and Orthogonal Arrays.
3. Redesign with Model-Based and Metamorphic Testing.

1. Changed Requirements

The new SwiftRide requirements are:

- Shared rides with 2–3 co-passengers.
- Dynamic surge multiplier from 1.0 to 3.0 in steps of 0.1.
- Surge based on zone and demand.
- Toll and airport charges.
- GST of 5%.
- Silver loyalty discount of 3%.
- Gold loyalty discount of 6%.
- Platinum loyalty discount of 10%.
- Time-window promotional codes.

- Approximately 35 decision points.
- Weekly releases.
- Full regression testing within 10 minutes.

Table 71: New Functional Requirements

Feature	**New Requirement**
Shared rides	2–3 co-passengers
Surge pricing	1.0–3.0
Surge increment	0.1
Toll	Additional charge
Airport	Additional charge
GST	5%
Silver	3% discount
Gold	6% discount
Platinum	10% discount
Time-window promo	Date/time-based eligibility
Regression time	Maximum 10 minutes

2. Extension of the Fare Calculation Model

The updated fare model must include the new charges and discounts.

For illustration, the following calculation order is assumed:

1. Calculate the base distance fare.
2. Add waiting charges.
3. Apply surge multiplier.
4. Add toll and airport charges.
5. Apply eligible promotional and loyalty discounts.

6. Add 5% GST to the discounted taxable amount.

The actual order must be confirmed by the business requirements because different calculation orders can produce different final fares.

The general formula is:

[
F_1=(B+W)\times S
]

Where:

- B = Base distance fare.
- W = Waiting charge.
- S = Surge multiplier.

Additional charges:

[
F_2=F_1+T+A
]

Where:

- T = Toll charge.
- A = Airport charge.

After discounts:

[
F_3=F_2-D
]

GST:

[
GST=F_3\times0.05
]

Final fare:

[
\boxed{Final\Fare=F_3+GST}
]

3. Example Fare Calculation

Assume:

- Base fare = ₹300.
- Waiting charge = ₹20.
- Surge multiplier = 1.5.
- Toll = ₹30.
- Airport charge = ₹50.
- Loyalty discount = Gold, 6%.
- GST = 5%.

Step 1: Apply surge.

[
F_1=(300+20)\times1.5
]

[
F_1=₹480
]

Step 2: Add toll and airport charges.

[
F_2=480+30+50
]

[
F_2=₹560
]

Step 3: Apply Gold discount.

[
D=560\times0.06
]

[
D=₹33.60
]

[
F_3=560-33.60
]

[
F_3=₹526.40
]

Step 4: Calculate GST.

[
GST=526.40\times0.05
]

[
GST=₹26.32
]

Final fare:

[
=526.40+26.32
]

[
\boxed{Final\ Fare=₹552.72}
]

This is an example using the assumed calculation order. The final implementation must follow the approved business specification.

4. Updated Fare Calculation Prototype

The following code demonstrates a simplified prototype of the new fare rules.

Program 5: Updated SwiftRide Fare Calculation

```
def calculate_new_fare(
    base_fare,
    wait_charge,
    surge,
    toll,
    airport,
    loyalty,
    promo_discount,
    passengers
):
    if passengers < 2 or passengers > 3:
        return "INVALID PASSENGER COUNT"
    if surge < 1.0 or surge > 3.0:
        return "INVALID SURGE"
    if round((surge - 1.0) * 10) != (surge - 1.0) * 10:
        return "INVALID SURGE STEP"
    if loyalty == "SILVER":
        discount = 0.03
    elif loyalty == "GOLD":
        discount = 0.06
    elif loyalty == "PLATINUM":
        discount = 0.10
```

```
    else:
        discount = 0
    fare = (base_fare + wait_charge) * surge
    fare = fare + toll + airport
    fare = fare - (fare * discount)
    fare = fare - promo_discount
    if fare < 0:
        fare = 0
    gst = fare * 0.05
    final_fare = fare + gst
    return round(final_fare, 2)
print(calculate_new_fare(
    300, 20, 1.5, 30, 50,
    "GOLD", 0, 2
))
```

Output:

552.72

The prototype demonstrates shared passenger validation, surge validation, loyalty discounts, additional charges and GST.

The time-window promo and zone-demand surge selection require additional input fields and approved business rules before they can be implemented completely.

5. Strategy A – Retain and Prioritise

The first strategy retains the existing Task 9 test suite.

Instead of executing every test after every small change, the tests are prioritised based on risk and the recently modified code.

High-priority tests include:

- Base fare calculation.
- Surge calculation.
- Shared passenger validation.
- GST calculation.
- Loyalty discount.
- Promo eligibility.
- Toll and airport charges.

Advantages:

- Reuses existing test cases.
- Requires less initial redesign.
- Supports faster testing after small changes.
- Suitable for stable functionality.

Limitations:

- New features may not be covered.
- Old test cases may use outdated assumptions.
- New interactions may remain untested.
- Regression confidence may decrease.

6. Strategy B – Modify with Decision Tables and Orthogonal Arrays

The second strategy modifies the existing suite to include new conditions and combinations.

The decision table is extended to include:

- Passenger count.
- Surge multiplier.
- Zone.

- Demand level.
- Toll.
- Airport charge.
- GST.
- Loyalty tier.
- Promo time window.

Orthogonal-array or pairwise testing is used to reduce the number of combinations.

Table 72: New Equivalence Classes

Parameter	Classes
Passenger count	2, 3, Invalid
Surge	1.0, 1.1–2.9, 3.0, Invalid
Zone	Normal, Airport, Restricted
Demand	Low, Medium, High
Toll	Zero, Positive
Airport	No, Yes
GST	5%
Loyalty	None, Silver, Gold, Platinum
Promo	Valid, Invalid, Expired

Advantages:

- Extends the existing test suite.
- Covers combinations of new business rules.
- Reduces the full Cartesian product.
- Maintains the existing boundary tests.

Limitations:

- Decision tables may become large.
- Some combinations may be difficult to maintain.
- Incorrect class definitions can miss faults.
- Pairwise coverage does not guarantee every higher-order interaction.

7. Strategy C – Redesign with Model-Based and Metamorphic Testing

The third strategy creates a new model of the fare calculation system.

Test cases are generated from the model, and the output is checked using both exact expected values and metamorphic relationships.

Metamorphic testing is useful when the exact expected result is difficult to calculate for every generated input.

The following relationships are considered.

Table 73: Metamorphic Properties

Property	**Expected Behaviour**
Increase distance	Fare should not decrease, assuming other inputs remain unchanged
Increase surge	Fare should not decrease
Add a toll	Fare should not decrease
Add airport charge	Fare should not decrease
Increase GST	Final fare should not decrease
Apply valid loyalty discount	Fare should decrease or remain equal
Apply expired promo	No promo discount should be applied
Invalid passenger count	System should reject the request

These properties must be checked with the approved rules. For example, shared rides may divide charges among passengers, so the comparison must specify whether it is checking total ride fare or each passenger's share.

Advantages:

- Supports complex fare models.
- Useful for frequently changing requirements.
- Detects unexpected relationships between inputs and outputs.
- Can generate large numbers of test cases.

Limitations:

- Requires a correct model.
- Metamorphic properties must be validated.
- More initial design effort is required.
- Incorrect assumptions can produce misleading results.

8. Impact Analysis of Task 9 Assumptions

The original Task 9 testing assumptions are no longer sufficient.

Table 74: Impact Analysis

Original Assumption	**Changed Requirement**	**Impact**
Small number of decisions	About 35 decisions	Higher control-flow complexity
Limited fare inputs	Many new parameters	More combinations
Stable pricing rules	Dynamic surge pricing	More frequent updates
No shared rides	2–3 co-passengers	New validation conditions
Simple promo rules	Time-window promos	Additional date/time tests
Fixed fare calculation	Toll, airport and GST	More calculation paths

Small regression suite	Weekly releases	Faster test maintenance required
Two-second test constraint	Ten-minute full run	Test selection and automation needed

9. Effect of Running the Old Suite Unchanged

The original Task 9 test suite can still test the old fare rules.

However, it does not fully cover:

- Shared passenger conditions.
- Surge pricing.
- Zone and demand combinations.
- Toll charges.
- Airport charges.
- GST.
- Loyalty discounts.
- Time-window promotions.

Therefore, the old suite may pass even when new features contain defects.

The old suite should be retained as a regression baseline, but additional tests are necessary.

10. Updated Cyclomatic Complexity

The changed requirement states that the new fare function contains approximately 35 decision points.

Using the simplified assumption that all 35 are independent predicate decisions:

[
V(G)=P+1
]

[
V(G)=35+1
]

[
\boxed{V(G)=36}
]

Therefore, the estimated number of independent basis paths is 36.

The exact complexity must be calculated from the final control flow graph. Compound conditions, short-circuit expressions and actual code structure may affect the count of predicate nodes.

11. Updated Basis-Path Testing

The new fare function requires tests for the independent paths introduced by the new conditions.

Important paths include:

- Normal shared ride.
- Invalid passenger count.
- Minimum surge.
- Maximum surge.
- Invalid surge increment.
- Toll applied.
- Airport charge applied.
- GST calculation.
- Silver discount.
- Gold discount.
- Platinum discount.
- Valid time-window promo.
- Expired promo.
- Invalid promo.
- High-demand surge.

- Low-demand surge.

A basis-path suite should include at least one test for each independent path, subject to feasibility.

12. Regression Runtime Calculation

The maximum regression time is 10 minutes.

Convert minutes into seconds:

[
10\times60=600\text{seconds}
]

If every test requires two seconds:

[
\text{Maximumtests}=\frac{600}{2}
]

[
\boxed{\text{Maximumtests}=300}
]

Therefore, if each test takes two seconds, the complete regression suite must contain no more than 300 tests to finish within 10 minutes.

This assumes sequential execution and excludes setup, teardown and reporting time. In practice, the suite should target fewer than 300 tests to provide a safety margin.

13. Comparison of the Three Adaptation Strategies

The following values are illustrative planning estimates, not measured experimental results.

Table 75: Adaptation Strategy Comparison

Criteria	**Retain and Prioritise**	**Modify Decision Tables**	**Model-Based and Metamorphic**
Test cases	50	100	120
New feature coverage	Low	High	High

Main advantage	Fast reuse	Better combination coverage	Better model-based validation
Maintenance effort	Low initially	Moderate	High initially
Fault detection	Limited for new features	Strong for rule combinations	Strong when model is accurate
Suitability for weekly releases	Moderate	High	High after model stabilisation

The test counts are proposed examples. Actual counts depend on the final requirements, test-generation tool and regression execution results.

14. Evaluation of the Original Suite

The original suite should be evaluated against the new features.

Table 76: Old Suite Coverage of New Requirements

New Feature	**Old Suite Coverage**	**Additional Tests Required**
Shared rides	Not covered	Yes
Surge pricing	Not covered	Yes
Toll	Not covered	Yes
Airport charge	Not covered	Yes
GST	Not covered	Yes
Loyalty discount	Not covered	Yes
Time-window promo	Not covered	Yes
Original fare slabs	Covered	Regression required

The old suite remains useful for checking the original fare calculation but cannot establish the correctness of the new functionality.

15. Evidence-Based Defence of the Final Decision

The final decision is:

<badge color="success">MODIFY</badge>

The existing test suite should be modified rather than completely discarded.

The reason is that the original fare calculation rules are still part of the new application. Removing all old test cases would waste useful regression coverage.

At the same time, simply retaining and prioritising the old suite would leave the new features untested.

Therefore, the recommended approach is:

1. Retain the original high-value regression tests.
2. Update the existing fare and boundary tests.
3. Extend the decision tables for new conditions.
4. Generate pairwise or orthogonal-array tests for combinations.
5. Add model-based tests for shared rides and fare calculation.
6. Add metamorphic checks for fare relationships.
7. Measure the complete regression runtime.
8. Remove redundant tests only after reviewing their coverage and risk.

16. Condition Under Which the Decision May Change

The decision may change from MODIFY to REDESIGN if the fare calculation is replaced by a substantially different pricing architecture or if the number of interacting decisions increases to a level where the existing test structure becomes difficult to maintain.

In that situation, a new model-based test suite may be more suitable.

17. Role of AI-Assisted Test Generation

AI-assisted test generation can help SwiftRide by:

- Generating test inputs.

- Suggesting boundary cases.
- Preparing decision tables.
- Creating test scripts.
- Identifying missing test scenarios.
- Suggesting regression test priorities.
- Explaining failed test results.

However, AI-generated tests must be reviewed because they may:

- Use incorrect fare calculation rules.
- Miss time-window conditions.
- Generate invalid combinations.
- Assume incorrect discount order.
- Ignore passenger-sharing rules.
- Produce expected outputs based on incorrect assumptions.

The final expected results must be verified against the approved requirements and an independent calculation method.

Result

The SwiftRide fare calculation system was adapted to include shared rides, surge pricing, additional charges, GST, loyalty discounts and time-window promotions.

Retain-and-prioritise, decision-table modification and model-based testing were compared.

The updated decision complexity is estimated at 36 under the stated 35-predicate assumption, and the 10-minute regression limit permits at most 300 sequential two-second tests before accounting for test overhead.

Final Conclusion

The MODIFY strategy is recommended for the updated SwiftRide system.

The existing regression suite should be retained and extended with new decision-table, pairwise, boundary and model-based tests.

This approach protects the original functionality while adding coverage for the new pricing rules. It also supports weekly releases and helps keep the complete regression suite within the required 10-minute execution limit.

OVERALL FINDINGS AND DISCUSSION

The complete testing process of the SwiftRide Cab Booking and Fare Engine helped demonstrate how software quality can be evaluated using different testing approaches. During the testing activities, different input values, business rules, program conditions, and execution paths were considered. This provided a better understanding of how errors can occur in different parts of an application and how suitable test cases can help identify them.

One of the major findings was that testing only the normal working conditions of an application is not sufficient. The system must also be tested with invalid inputs, boundary values, unexpected combinations, and different user actions. For example, in fare calculation, a small difference in travel distance, waiting time, or booking time can affect the final fare. Similarly, an incorrect promotional condition or wallet balance check may result in an incorrect transaction. Therefore, testing different situations is necessary to ensure that the system behaves correctly.

Another important finding was that each testing technique provides a different level of understanding of the application. Black-box testing focuses on the expected behaviour of the system, whereas white-box testing examines its internal structure. Boundary Value Analysis is useful for identifying errors around limits, while decision table testing helps verify combinations of business conditions. State transition testing checks whether the booking process follows the correct sequence of operations. Pairwise testing helps reduce the number of test cases when multiple input parameters are involved.

The code analysis activities also showed the importance of measuring test effectiveness. Statement and branch coverage indicate how much of the program has been executed, but they do not guarantee that all defects have been detected. Mutation testing provides additional information by checking whether the test cases can identify small changes in the program. Similarly, Control Flow Graphs and Cyclomatic Complexity help understand the program structure and identify independent execution paths that require testing.

The comparison of different test suites highlighted the importance of selecting test cases based on risk, functionality, and system complexity. A large number of test cases does not

automatically mean better testing. Instead, test cases should be meaningful, cover important conditions, and identify possible defects without unnecessary repetition. Combining multiple testing techniques can provide better confidence in the quality of the application.

CHALLENGES FACED DURING TESTING

Several challenges were considered during the testing of the SwiftRide system. One challenge was identifying all the possible combinations of input conditions. The fare engine contains multiple parameters, including distance, waiting time, booking time, promotional codes, payment modes, and wallet balance. Testing every possible combination would require a large number of test cases. Pairwise testing and decision table techniques help manage this complexity.

Another challenge was identifying the correct expected output for each test case. Fare calculation involves multiple operations, such as base fare calculation, waiting charges, night surcharges, discounts, and fare limits. The order of these operations can affect the final result. Therefore, the business rules must be clearly understood before preparing expected results.

Testing the internal program structure also requires careful analysis. A test suite may execute every statement but still fail to detect certain logical errors. Identifying independent paths, evaluating individual conditions, and analysing surviving mutants require additional effort. It is also important to distinguish equivalent mutants from actual faults because some program changes may not affect the final output.

Changing requirements create another challenge. When new features such as shared rides, surge pricing, loyalty discounts, and additional charges are introduced, existing test cases may no longer be sufficient. The testing team must identify affected areas, update test data, and perform regression testing to ensure that existing functionality continues to work correctly.

IMPORTANCE OF TESTING IN REAL-WORLD APPLICATIONS

Software testing is essential in real-world applications because users expect systems to provide correct and consistent results. In a cab booking application, incorrect fare calculation can create payment problems and reduce customer trust. Incorrect driver searching or booking status updates can also affect the overall user experience.

A proper testing process helps identify such issues before the application is released. It also reduces the possibility of failures during actual usage. By testing valid and invalid inputs, developers can improve input validation and make the application more reliable.

Testing also supports future software maintenance. When developers introduce new features or modify existing business rules, regression testing helps verify that the previous functionality has not been affected. Automated testing can further reduce repeated manual effort and provide faster feedback during development.

FUTURE ENHANCEMENTS

The testing process can be improved further by introducing more automated testing methods. Automated test scripts can execute frequently used test cases and help verify the system after every software update. This can reduce manual work and make regression testing faster.

The test suite can also be expanded to include performance testing, security testing, and usability testing. Performance testing can evaluate how the system behaves when many users book rides at the same time. Security testing can help identify weaknesses related to user information, payment details, and account access. Usability testing can examine whether users can easily complete the booking and payment process.

In addition, data-driven testing can be used to execute the same test logic with different sets of input values. Risk-based testing can give higher priority to important features such as fare calculation, payment processing, and booking confirmation. Model-based testing can also help generate test cases from the booking lifecycle and its state transitions.

Artificial intelligence tools may support test case generation and defect identification. However, generated test cases must be reviewed and validated before they are included in the final test suite. Human understanding of business requirements remains important for ensuring that the expected behaviour is correct.

FINAL REMARKS

The overall testing activities demonstrate that software testing is a continuous process that supports the development and maintenance of reliable applications. Applying different testing techniques helps examine the system from multiple perspectives and improves the chances of detecting defects.

The SwiftRide Cab Booking and Fare Engine served as a practical example for understanding how testing concepts can be applied to a real-world system. The assignment strengthened the understanding of test case design, code coverage, mutation testing, decision logic, control flow analysis, and regression testing.

A successful testing strategy should focus not only on the number of test cases but also on their effectiveness, accuracy, and ability to detect faults. By combining suitable testing techniques and updating the test suite whenever requirements change, software quality can be improved and the system can provide more reliable results to its users.